Automatic Product-Image Tagging

Which neural network should tag the fashion catalogue, and how much of the work can it take over?

A stakeholder report comparing four deep-learning variations - a dense network and three convolutional neural networks - for classifying product images, with a recommended model, what it gets right and wrong, and how to deploy it safely.

93.7%

test accuracy of the recommended CNN on 10,000 unseen product images

80%

of images can be tagged automatically at 99.1% accuracy (confidence >= 0.95)

41%

fewer wrong tags than a standard dense neural network

Prepared for	Chief Data Officer and Head of Analytics
Prepared by	Omar Essam, Data Scientist
Date	October 24, 2026
Analysis behind this report	Final Project/Deep Learning - Fashion Product Image Classification.ipynb

Contents

Executive summary 2
1. Main objective of the analysis 3
2. Data description 3
3. Data exploration and preparation 4
4. Model variations 4
5. Recommended final model 5
6. Key findings and insights 6
7. Limitations of the model 8
8. Next steps 9
Appendix: training set-up and reproducibility 10

Executive summary

An online fashion retailer adds thousands of product photos to its catalogue every week, and each one has to be tagged with its category before it can be searched, filtered and recommended. This analysis trains deep-learning models that read a product image and predict its category, and asks how much of the tagging work they can safely automate.

Key insights

- **Convolutions matter.** The recommended CNN reaches 93.7% test accuracy against 89.3% for a dense network that treats pixels independently - 41% fewer wrong tags.
- **Regularised depth wins.** Batch normalisation and dropout let the deeper CNN reach the best accuracy with a small train-test gap (2.7 points). Data augmentation shrank the gap further (1.3 points) but was still improving when the 40-epoch CPU budget ran out.
- **Most products are easy; tops are hard.** Bag, Trouser, Sandal are recognised almost perfectly, while Shirt is the weakest class (recall 79%); the most common mix-up is Shirt tagged as T-shirt/top (78 of 1,000 test images).
- **Confidence makes automation safe.** Accepting only predictions with confidence of at least 0.95 tags 80% of images at 99.1% accuracy; the remaining 20% go to a human tagger.

Recommendation

- **Deploy the Deep CNN (BatchNorm + Dropout) as a human-in-the-loop tagger:** auto-accept tags above confidence 0.95 and route the rest to the catalogue team. It is the most accurate variation (93.7%, macro F1 0.937), remains small (288,682 parameters) and tags an image in about 0.22 ms on an ordinary CPU.
- **Next:** retrain on the retailer's own colour photos with transfer learning, and add product text (title, description) to separate shirts from tops.

1. Main objective of the analysis

The objective is to automate the tagging of product images with supervised deep learning, specifically convolutional neural networks (CNNs) for multi-class image classification. The model has to predict which of ten categories a product image belongs to, as accurately as possible on images it has never seen, and it has to say how sure it is, so that the business can decide which tags to trust automatically. The focus is on prediction accuracy, supported by explanations (confusion analysis, learned filters and Grad-CAM heatmaps) that show stakeholders why the model can be trusted and where it cannot.

The benefits for the business are:

- **Faster time to market:** new products are searchable as soon as their photo is uploaded.
- **Lower tagging cost:** people only review the uncertain images instead of every image.
- **Better catalogue data quality:** consistent categories improve search, filters, recommendations and reporting.
- **A reusable capability:** the same approach can flag mislabelled products already in the catalogue.

2. Data description

The data is **Fashion-MNIST**, published by Zalando Research as a public benchmark built from Zalando's own product photos: 60,000 training and 10,000 test images of articles from ten categories, exactly 7,000 per category. It stands in for the retailer's catalogue images.

Element	Description	Use in the deep-learning task
Image	28 x 28 pixels, grayscale, the product centred on a white background	Input of the network (one channel)
Pixel values	784 intensities per image, 0 (white) to 255 (black)	Scaled to 0-1; local patterns of pixels are what the convolutions learn
Label	10 categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot	Target of the softmax output layer
Split	54,000 training, 6,000 validation (from the official training set), 10,000 test	Training, early stopping, final comparison

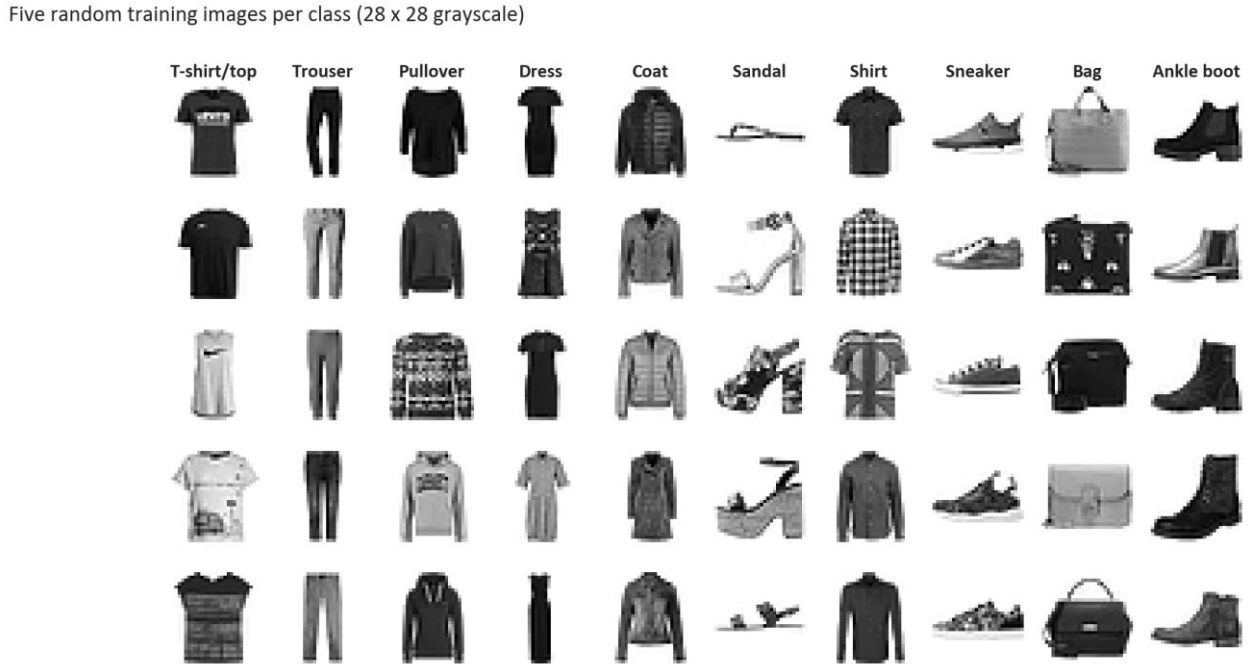


Figure 1 - Five random training images of each category. Shirts, T-shirts/tops, pullovers and coats look alike at this resolution.

The analysis aims to find the deep-learning architecture that classifies these images most accurately, understand its mistakes, and turn its confidence scores into a practical automation rule.

3. Data exploration and preparation

- The classes are perfectly balanced (6,000 training and 1,000 test images each), so accuracy is a fair headline metric; macro F1 is reported as well.
- Data quality: no missing values, no exact duplicate images inside the training set and no test image that also occurs in the training set (checked by hashing every image), so there is no leakage between training and evaluation.
- Mean pixel intensity differs by class (Coat is the darkest), but shape and texture - not brightness - separate the confusable tops, which motivates convolutions.
- Preparation: pixels scaled to 0-1 and reshaped to 28 x 28 x 1; a stratified 6,000-image validation set was held out of the official training images for early stopping, and the official test set was kept untouched for the final comparison.
- Feature engineering is learned by the network itself: each convolution kernel is a small 3 x 3 grid of weights slid across the image to detect a local pattern such as an edge or a texture. Without padding a 3 x 3 kernel turns a 28 x 28 image into 26 x 26 (and a 3 x 3 image into a single value); 'same' padding keeps 28 x 28 so the borders are not lost.
- Data augmentation (variation 4) creates new training views on the fly - mirror images, shifts and zooms of up to 8% - so the network learns that a product is the same product wherever it sits in the frame.

4. Model variations

Four variations were trained on the same 54,000 images with the same optimiser (Adam, learning rate 0.001, batch 128), early stopping on validation loss (patience 5, best weights kept) and learning-rate halving on plateaus:

Variation	Architecture
Dense network (MLP)	Flatten - Dense 256 - Dropout 0.3 - Dense 128 - Dropout 0.3 - softmax
Small CNN	Conv 32 (3x3) - MaxPool - Conv 64 (3x3) - MaxPool - Dense 128 - Dropout 0.3 - softmax
Deep CNN (BatchNorm + Dropout)	3 VGG-style blocks: 2 x Conv 32, 2 x Conv 64, Conv 128 (3x3, BatchNorm, ReLU), MaxPool + Dropout 0.25 after each; Dense 128 - Dropout 0.4 - softmax
Deep CNN + data augmentation	Same deep CNN, trained on images randomly flipped left-right, shifted and zoomed by up to 8%

Variation	Parameters	Epochs	Train min	Train acc	Test acc [95% CI]	Macro F1	ms / image
Dense network (MLP)	235,146	27	1.0	92.9%	89.3% [88.7%-89.9%]	0.893	0.03
Small CNN	421,642	14	2.2	94.7%	91.6% [91.0%-92.2%]	0.916	0.06
Deep CNN (BatchNorm + Dropout)	288,682	27	23.5	96.4%	93.7% [93.2%-94.1%]	0.937	0.22
Deep CNN + data augmentation	288,682	40	32.8	94.0%	92.6% [92.1%-93.1%]	0.926	0.21

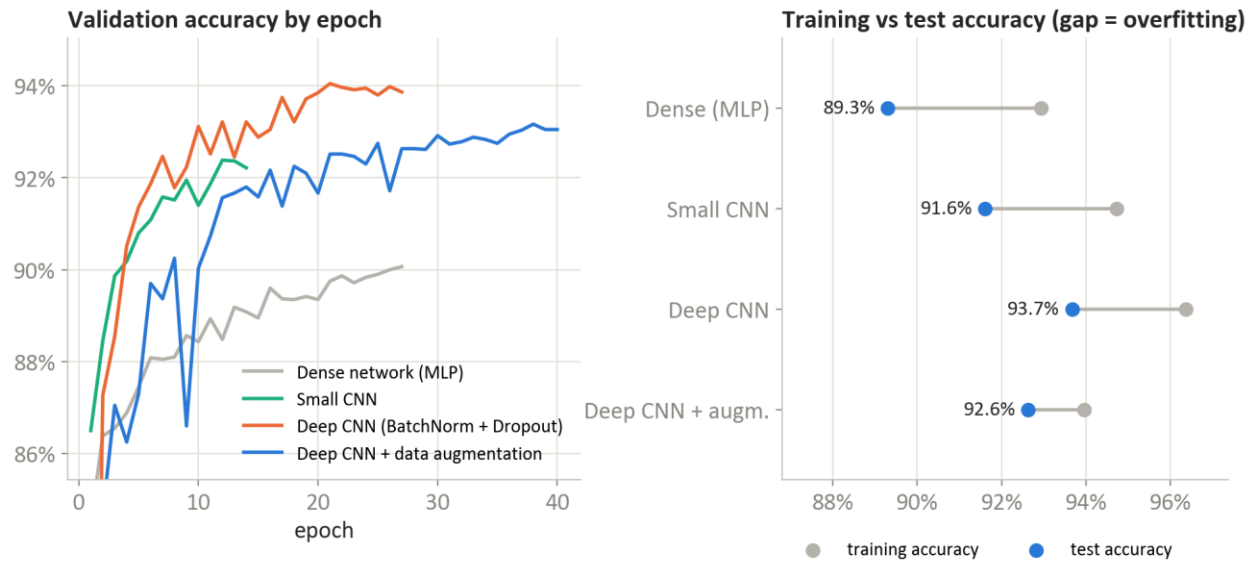


Figure 2 - Validation accuracy during training (left) and training vs test accuracy of each variation (right); a wide gap between the gray and blue dots means the model memorised the training images.

- **Dense network:** 89.3% test accuracy. Flattening the image throws away which pixels are neighbours, and it overfits the most (training accuracy 92.9%).
- **Small CNN:** 91.6% with far fewer epochs: two convolution blocks already capture edges and textures, but the train-test gap (3.1 points) shows it starts to memorise.
- **Deep CNN with batch normalisation and dropout:** 93.7%; more capacity, while dropout and batch normalisation keep the gap at 2.7 points.
- **Deep CNN with data augmentation:** 92.6%; augmentation is the strongest regulariser (gap 1.3 points), but harder training images make learning slower: it used all 40 epochs, its best epoch was 38 and validation accuracy was still rising - it is under-trained rather than worse, and the longest to train (33 minutes on a CPU).

The 95% bootstrap interval of the accuracy difference between the best variation (Deep CNN (BatchNorm + Dropout)) and the runner-up (Deep CNN + data augmentation) is +0.0063 to +0.0147 - the ranking is statistically clear.

5. Recommended final model

Recommended: the Deep CNN (BatchNorm + Dropout). It best fits the objective of accurate, trustworthy automatic tagging:

- **Most accurate:** 93.7% test accuracy and macro F1 0.937, the best of the four variations.
- **Generalises well:** training and test accuracy differ by only 2.7 points, so performance on new products should hold up.
- **Cheap to run:** 288,682 parameters and about 0.22 ms per image on a CPU - no GPU is needed to tag the catalogue.
- **Explainable enough for the business:** its confidence scores are reliable for routing (section 6), and Grad-CAM heatmaps show that it reads each product's silhouette - sleeves, hems, handles and outline.

The dense network is simpler to explain mathematically but makes many more mistakes, so it is not recommended; the small CNN is a reasonable fallback where training time matters more than the last point of accuracy.

6. Key findings and insights

Where the model is right and wrong

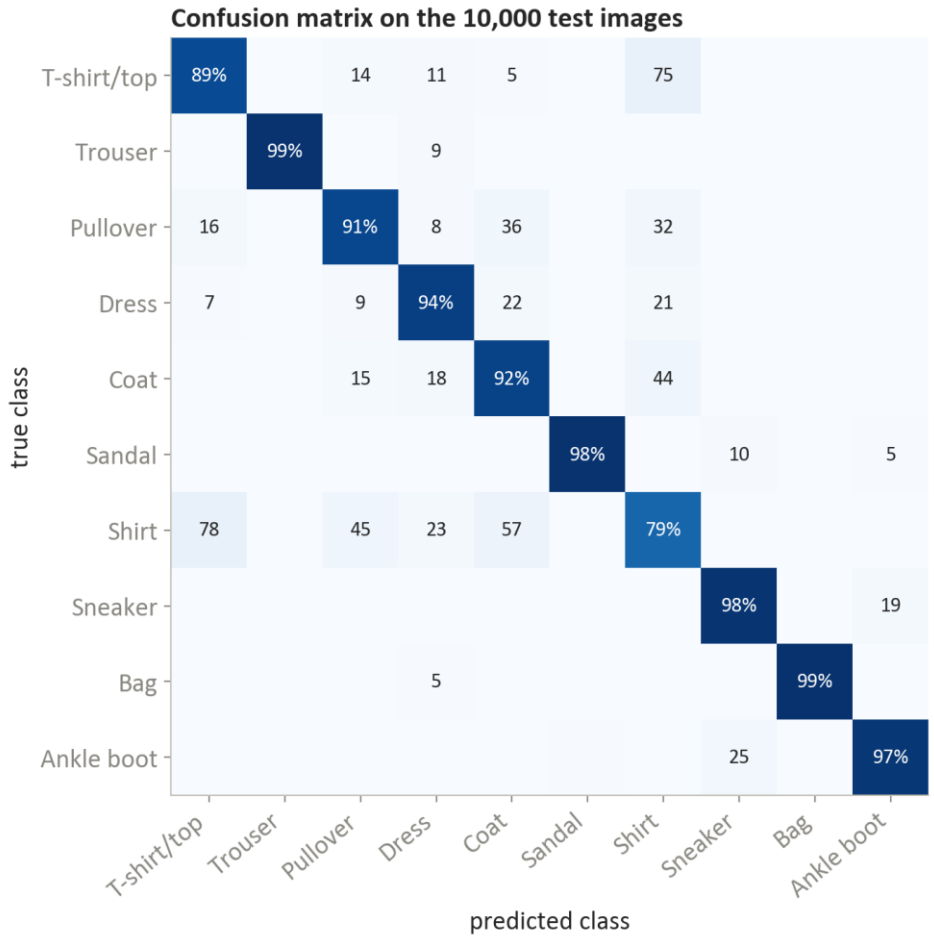


Figure 3 - Confusion matrix of the recommended model (diagonal = recall per class; off-diagonal = number of test images).

Class	Precision	Recall	F1
Shirt	82.0%	79.3%	0.806
T-shirt/top	89.7%	89.3%	0.895
Pullover	91.5%	90.7%	0.911
Coat	88.4%	92.3%	0.903
Dress	92.7%	94.1%	0.934
Ankle boot	97.6%	97.0%	0.973
Sneaker	96.5%	97.9%	0.972
Sandal	99.1%	98.5%	0.988
Trouser	99.7%	98.6%	0.991
Bag	99.4%	99.0%	0.992

- Footwear, bags and trousers are recognised almost perfectly: their silhouettes are distinctive.
- Upper-body garments are the hard group. Shirt has the lowest recall (79.3%); the most frequent confusions are Shirt tagged as T-shirt/top (78); T-shirt/top tagged as Shirt (75); Shirt tagged as Coat (57).
- **The most confident mistakes point to label errors, not model errors (Figure 4):** images labelled 'Sneaker' that are clearly high-top boots, a 'T-shirt/top' that is a sleeveless dress, a 'Shirt' that looks like a bag. In these cases the model's answer is arguably the right one, so part of the measured error sits in the data, and the model can be used to find mislabelled products.



Figure 4 - The ten mistakes made with the highest confidence (true label, predicted label, confidence). Most look mislabelled rather than misread.

How much tagging can be automated

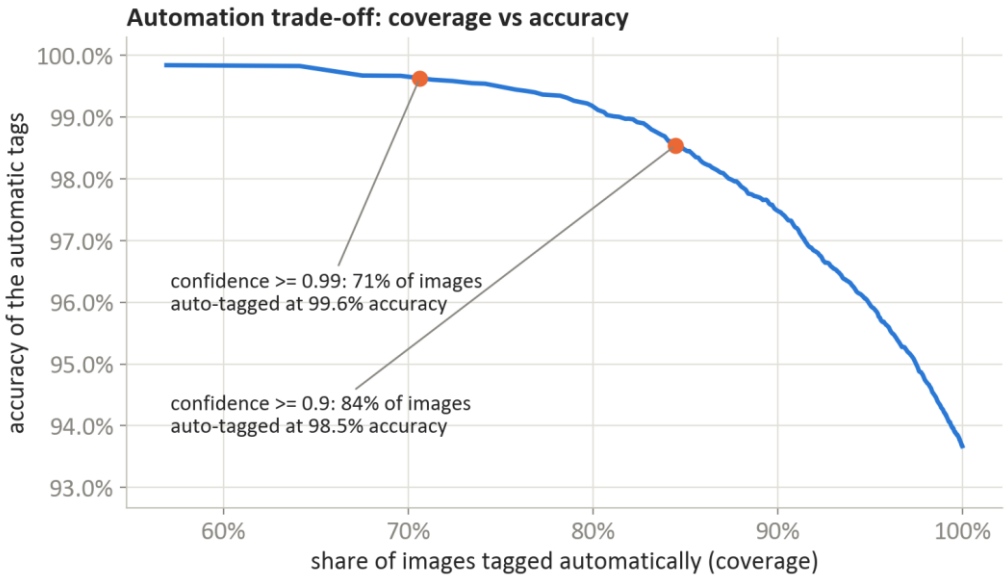


Figure 5 - If only predictions above a confidence threshold are accepted automatically, accuracy rises while coverage falls (two thresholds marked; the table below lists more).

Confidence threshold	Tagged automatically	Accuracy of automatic tags	Sent to human review
none (accept all)	100.0%	93.67%	0.0%
0.7	92.8%	96.64%	7.2%
0.9	84.5%	98.54%	15.5%
0.95	80.1%	99.14%	19.9%
0.99	70.6%	99.63%	29.4%

At a threshold of 0.95, 80% of new product images are tagged without human effort at 99.1% accuracy, and the catalogue team only reviews 20%. A looser threshold of 0.9 automates 84% at 98.5%. The threshold is a business decision that trades review cost against the cost of a wrong tag.

What the network has learned

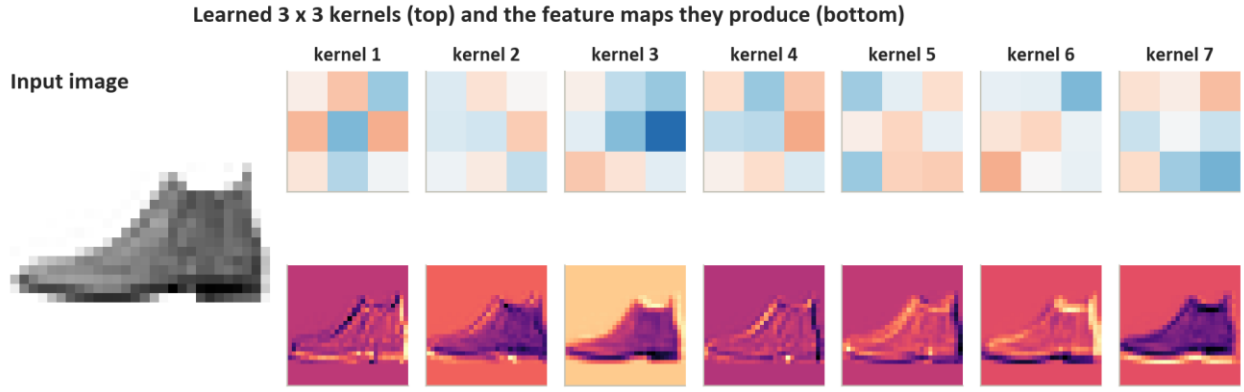


Figure 6 - Seven of the first-layer 3 x 3 kernels (top) and the feature maps they produce for one ankle boot (bottom): each kernel responds to a different edge orientation or texture.

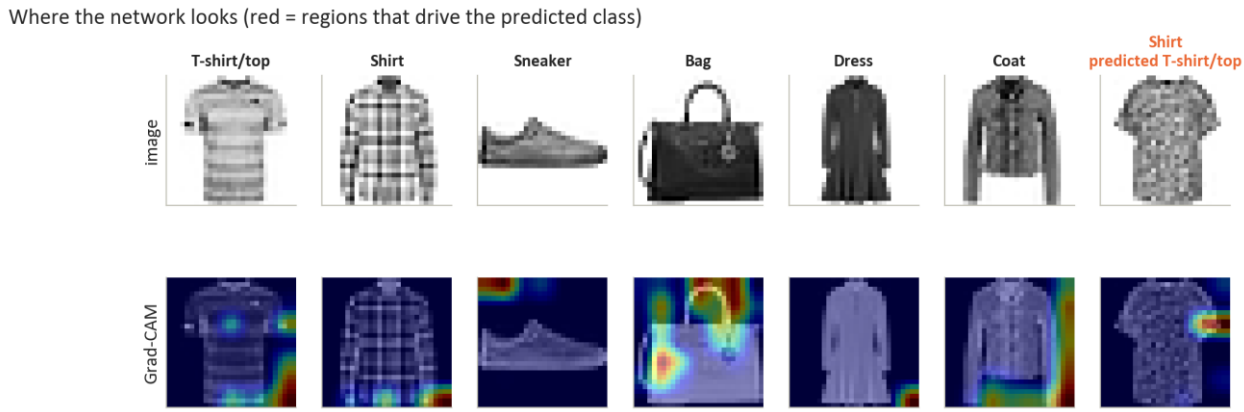


Figure 7 - Grad-CAM heatmaps of the last convolution layer: the regions that drive each prediction (red). The model reads outlines - sleeve ends, hems, handles - and the empty space around them.

The first convolution layer has learned edge and texture detectors - the 'kernels' of a CNN - and deeper layers combine them into parts of the silhouette. Grad-CAM shows that the decisive evidence is the outline: the ends of sleeves, the hem, the handle of a bag, and for sneakers and dresses the empty space around the product (a low shoe leaves the top of the frame empty, which separates it from an ankle boot). In the misclassified shirt the model looks at the short sleeve, which is exactly why it reads the item as a T-shirt. Reasoning from silhouettes works on clean studio images, but it is also a risk for photos with busy backgrounds (section 7).

7. Limitations of the model

- **Benchmark images, not the real catalogue:** 28 x 28 grayscale photos on clean backgrounds are much simpler than real colour photos with models, props and varying light; accuracy will drop on production images until the model is retrained on them.
- **Only ten broad categories:** a real catalogue has hundreds of categories and attributes (colour, sleeve length, pattern); the model cannot tag what it has not been trained on.
- **Ambiguous classes and noisy labels:** the Shirt / T-shirt boundary is inherently fuzzy at this resolution, and several of the most confident 'mistakes' are mislabelled images, so the test score is a noisy measure of the true quality.
- **Reliance on silhouette and background:** Grad-CAM shows the model uses the empty space around a product as evidence; cluttered or cropped production photos could mislead it.

- **Confidence is not certainty:** 19% of the mistakes are still made with confidence of at least 0.9, so automatic tags need periodic audits.
- **Single training run and CPU budget:** each variation was trained once with one random seed, the augmented model was stopped by the epoch cap while still improving, and architectures and hyper-parameters were not searched exhaustively.

8. Next steps

- **Transfer learning on real photos:** fine-tune a pre-trained network (for example EfficientNet or ResNet) on the retailer's own colour images at higher resolution.
- **Add product text:** combine the image with the title and description in a multimodal model - words such as 'button-down' or 'collar' resolve the shirt vs T-shirt confusion.
- **Calibrate and monitor confidence:** apply temperature scaling, track accuracy of auto-accepted tags in production, and adjust the threshold to the cost of errors.
- **Clean the labels with the model:** review the most confident mistakes to fix mislabelled images, and collect more examples of the confusable tops.
- **Finish the augmented model, then tune and ensemble:** train the augmented CNN to convergence on a GPU (it was still improving), search learning rates, depth and augmentation strength, and average several CNNs.
- **Pilot in the catalogue workflow:** start with one product line, compare tagging time and error rates with the manual process, then scale.

Appendix: training set-up and reproducibility

Setting	Value
Optimiser	Adam, learning rate 0.001, halved after 2 epochs without validation improvement (minimum 0.00001)
Batch size / epochs	128 / up to 30-40 with early stopping on validation loss (patience 5, best weights restored)
Loss and output	sparse categorical cross-entropy, 10-way softmax
Regularisation	dropout 0.25-0.4, batch normalisation (deep CNNs), random flip / shift / zoom (variation 4)
Evaluation	official 10,000-image test set; 95% bootstrap intervals from 1,000 resamples

- Data: Fashion-MNIST (github.com/zalandoresearch/fashion-mnist), loaded with keras.datasets.fashion_mnist.
- Analysis: Final Project/Deep Learning - Fashion Product Image Classification.ipynb - every table and figure in this report is produced there and saved to the results and figures folders.
- Environment: Python 3.11, TensorFlow 2.17 / Keras 3, scikit-learn 1.5, matplotlib 3.9; trained on CPU; random seed 123.
- This report: Final Project/source/build_report.py builds it from the saved results; export_pdf.ps1 updates the contents and saves the PDF with Microsoft Word.